

rcore

// Window-related functions

```
void InitWindow(int width, int height, const char *title); // Initialize window and OpenGL context
void CloseWindow(void); // Close window and unload OpenGL context
bool WindowShouldClose(void); // Check if application should close (KEY_ESCAPE pressed or windows close icon clicked)
bool IsWindowReady(void); // Check if window has been initialized successfully
bool IsWindowFullscreen(void); // Check if window is currently fullscreen
bool IsWindowHidden(void); // Check if window is currently hidden
bool IsWindowMinimized(void); // Check if window is currently minimized
bool IsWindowMaximized(void); // Check if window is currently maximized
bool IsWindowFocused(void); // Check if window is currently focused
bool IsWindowResized(void); // Check if window has been resized last frame
bool IsWindowState(unsigned int flag); // Check if one specific window flag is enabled
void SetWindowState(unsigned int flags); // Set window configuration state using flags
void ClearWindowState(unsigned int flags); // Clear window configuration state flags
void ToggleFullscreen(void); // Toggle window state: fullscreen/windowed, resizes monitor to match window resolution
void ToggleBorderlessWindowed(void); // Toggle window state: borderless windowed, resizes window to match monitor resolution
void MaximizeWindow(void); // Set window state: maximized, if resizable
void MinimizeWindow(void); // Set window state: minimized, if resizable
void RestoreWindow(void); // Restore window from being minimized/maximized
void SetWindowIcon(Image image); // Set icon for window (single image, RGBA 32bit)
void SetWindowIcons(Image *images, int count); // Set icon for window (multiple images, RGBA 32bit)
void SetWindowTitle(const char *title); // Set title for window
void SetWindowPosition(int x, int y); // Set window position on screen
void SetWindowMonitor(int monitor); // Set monitor for the current window
void SetWindowMinSize(int width, int height); // Set window minimum dimensions (for FLAG_WINDOW_RESIZABLE)
void SetWindowMaxSize(int width, int height); // Set window maximum dimensions (for FLAG_WINDOW_RESIZABLE)
void SetWindowSize(int width, int height); // Set window dimensions
void SetWindowOpacity(float opacity); // Set window opacity [0.0f..1.0f]
void SetWindowFocused(void); // Set window focused
void *GetWindowHandle(void); // Get native window handle
int GetScreenWidth(void); // Get current screen width
int GetScreenHeight(void); // Get current screen height
int GetRenderWidth(void); // Get current render width (it considers HiDPI)
int GetRenderHeight(void); // Get current render height (it considers HiDPI)
int GetMonitorCount(void); // Get number of connected monitors
int GetCurrentMonitor(void); // Get current monitor where window is placed
Vector2 GetMonitorPosition(int monitor); // Get specified monitor position
int GetMonitorWidth(int monitor); // Get specified monitor width (current video mode used by monitor)
int GetMonitorHeight(int monitor); // Get specified monitor height (current video mode used by monitor)
int GetMonitorPhysicalWidth(int monitor); // Get specified monitor physical width in millimetres
int GetMonitorPhysicalHeight(int monitor); // Get specified monitor physical height in millimetres
int GetMonitorRefreshRate(int monitor); // Get specified monitor refresh rate
Vector2 GetWindowPosition(void); // Get window position XY on monitor
Vector2 GetWindowScaleDPI(void); // Get window scale DPI factor
const char *GetMonitorName(int monitor); // Get the human-readable, UTF-8 encoded name of the specified monitor
void SetClipboardText(const char *text); // Set clipboard text content
const char *GetClipboardText(void); // Get clipboard text content
Image GetClipboardImage(void); // Get clipboard image content
void EnableEventWaiting(void); // Enable waiting for events on EndDrawing(), no automatic event polling
void DisableEventWaiting(void); // Disable waiting for events on EndDrawing(), automatic events polling
```

// Cursor-related functions

```
void ShowCursor(void); // Shows cursor
void HideCursor(void); // Hides cursor
bool IsCursorHidden(void); // Check if cursor is not visible
void EnableCursor(void); // Enables cursor (unlock cursor)
void DisableCursor(void); // Disables cursor (lock cursor)
bool IsCursorOnScreen(void); // Check if cursor is on the screen
```

// Drawing-related functions

```
void ClearBackground(Color color); // Set background color (framebuffer clear color)
void BeginDrawing(void); // Setup canvas (framebuffer) to start drawing
void EndDrawing(void); // End canvas drawing and swap buffers (double buffering)
```

```

void BeginMode2D(Camera2D camera);           // Begin 2D mode with custom camera (2D)
void EndMode2D(void);                         // Ends 2D mode with custom camera
void BeginMode3D(Camera3D camera);           // Begin 3D mode with custom camera (3D)
void EndMode3D(void);                         // Ends 3D mode and returns to default 2D orthographic mode
void BeginTextureMode(RenderTexture2D target); // Begin drawing to render texture
void EndTextureMode(void);                   // Ends drawing to render texture
void BeginShaderMode(Shader shader);         // Begin custom shader drawing
void EndShaderMode(void);                   // End custom shader drawing (use default shader)
void BeginBlendMode(int mode);               // Begin blending mode (alpha, additive, multiplied, subtract, custom)
void EndBlendMode(void);                   // End blending mode (reset to default: alpha blending)
void BeginScissorMode(int x, int y, int width, int height); // Begin scissor mode (define screen area for following drawing)
void EndScissorMode(void);                 // End scissor mode
void BeginVrStereoMode(VrStereoConfig config); // Begin stereo rendering (requires VR simulator)
void EndVrStereoMode(void);                 // End stereo rendering (requires VR simulator)

// VR stereo config functions for VR simulator
VrStereoConfig LoadVrStereoConfig(VrDeviceInfo device); // Load VR stereo config for VR simulator device parameters
void UnloadVrStereoConfig(VrStereoConfig config);       // Unload VR stereo config

// Shader management functions
// NOTE: Shader functionality is not available on OpenGL 1.1
Shader LoadShader(const char *vsFileName, const char *fsFileName); // Load shader from files and bind default locations
Shader LoadShaderFromMemory(const char *vsCode, const char *fsCode); // Load shader from code strings and bind default locations
bool IsShaderValid(Shader shader); // Check if a shader is valid (loaded on GPU)
int GetShaderLocation(Shader shader, const char *uniformName); // Get shader uniform location
int GetShaderLocationAttrib(Shader shader, const char *attribName); // Get shader attribute location
void SetShaderValue(Shader shader, int locIndex, const void *value, int uniformType); // Set shader uniform value
void SetShaderValueV(Shader shader, int locIndex, const void *value, int uniformType, int count); // Set shader uniform value vector
void SetShaderValueMatrix(Shader shader, int locIndex, Matrix mat); // Set shader uniform value (matrix 4x4)
void SetShaderValueTexture(Shader shader, int locIndex, Texture2D texture); // Set shader uniform value and bind the texture (sampler2d)
void UnloadShader(Shader shader); // Unload shader from GPU memory (VRAM)

// Screen-space-related functions
Ray GetScreenToWorldRay(Vector2 position, Camera camera); // Get a ray trace from screen position (i.e mouse)
Ray GetScreenToWorldRayEx(Vector2 position, Camera camera, int width, int height); // Get a ray trace from screen position (i.e mouse) in a viewport
Vector2 GetWorldToScreen(Vector3 position, Camera camera); // Get the screen space position for a 3d world space position
Vector2 GetWorldToScreenEx(Vector3 position, Camera camera, int width, int height); // Get size position for a 3d world space position
Vector2 GetWorldToScreen2D(Vector2 position, Camera2D camera); // Get the screen space position for a 2d camera world space position
Vector2 GetScreenToWorld2D(Vector2 position, Camera2D camera); // Get the world space position for a 2d camera screen space position
Matrix GetCameraMatrix(Camera camera); // Get camera transform matrix (view matrix)
Matrix GetCameraMatrix2D(Camera2D camera); // Get camera 2d transform matrix

// Timing-related functions
void SetTargetFPS(int fps); // Set target FPS (maximum)
float GetFrameTime(void); // Get time in seconds for last frame drawn (delta time)
double GetTime(void); // Get elapsed time in seconds since InitWindow()
int GetFPS(void); // Get current FPS

// Custom frame control functions
// NOTE: Those functions are intended for advanced users that want full control over the frame processing
// By default EndDrawing() does this job: draws everything + SwapScreenBuffer() + manage frame timing + PollInputEvents()
// To avoid that behaviour and control frame processes manually, enable in config.h: SUPPORT_CUSTOM_FRAME_CONTROL
void SwapScreenBuffer(void); // Swap back buffer with front buffer (screen drawing)
void PollInputEvents(void); // Register all input events
void WaitTime(double seconds); // Wait for some time (halt program execution)

// Random values generation functions
void SetRandomSeed(unsigned int seed); // Set the seed for the random number generator
int GetRandomValue(int min, int max); // Get a random value between min and max (both included)
int *LoadRandomSequence(unsigned int count, int min, int max); // Load random values sequence, no values repeated
void UnloadRandomSequence(int *sequence); // Unload random values sequence

```

```

// Misc. functions
void TakeScreenshot(const char *fileName); // Takes a screenshot of current screen (filename extension defines format)
void SetConfigFlags(unsigned int flags); // Setup init configuration flags (view FLAGS)
void OpenURL(const char *url); // Open URL with default system browser (if available)

// Logging system
void SetTraceLogLevel(int logLevel); // Set the current threshold (minimum) log level
void TraceLog(int logLevel, const char *text, ...); // Show trace log messages (LOG_DEBUG, LOG_INFO, LOG_WARNING, LOG_ERROR...)
void SetTraceLogCallback(TraceLogCallback callback); // Set custom trace log

// Memory management, using internal allocators
void *MemAlloc(unsigned int size); // Internal memory allocator
void *MemRealloc(void *ptr, unsigned int size); // Internal memory reallocator
void MemFree(void *ptr); // Internal memory free

// File system management functions
unsigned char *LoadFileData(const char *fileName, int *dataSize); // Load file data as byte array (read)
void UnloadFileData(unsigned char *data); // Unload file data allocated by LoadFileData()
bool SaveFileData(const char *fileName, void *data, int dataSize); // Save data to file from byte array (write), returns true on success
bool ExportDataAsCode(const unsigned char *data, int dataSize, const char *fileName); // Export text data to code (.h), returns true on success
char *LoadFileText(const char *fileName); // Load text data from file (read), returns a '\0' terminated string
void UnloadFileText(char *text); // Unload file text data allocated by LoadFileText()
bool SaveFileText(const char *fileName, const char *text); // Save text data to file (write), string must be '\0' terminated, returns true on success

// File access custom callbacks
// WARNING: Callbacks setup is intended for advanced users
void SetLoadFileDataCallback(LoadFileDataCallback callback); // Set custom file binary data loader
void SetSaveFileDataCallback(SaveFileDataCallback callback); // Set custom file binary data saver
void SetLoadFileTextCallback(LoadFileTextCallback callback); // Set custom file text data loader
void SetSaveFileTextCallback(SaveFileTextCallback callback); // Set custom file text data saver
int FileRename(const char *fileName, const char *fileRename); // Rename file (if exists)
int FileRemove(const char *fileName); // Remove file (if exists)
int FileCopy(const char *srcPath, const char *dstPath); // Copy file from one path to another, dstPath created if it doesn't exist
int FileMove(const char *srcPath, const char *dstPath); // Move file from one directory to another, dstPath created if it doesn't exist
int FileTextReplace(const char *fileName, const char *search, const char *replacement); // Replace text in an existing file
int FileTextFindIndex(const char *fileName, const char *search); // Find text in existing file
bool FileExists(const char *fileName); // Check if file exists
bool DirectoryExists(const char *dirPath); // Check if a directory path exists
bool IsFileExtension(const char *fileName, const char *ext); // Check file extension (recommended include point: .png, .wav)
int GetFileLength(const char *fileName); // Get file length in bytes (NOTE: GetFileSize() conflicts with windows.h)
long GetFileModTime(const char *fileName); // Get file modification time (last write time)
const char *GetFileExtension(const char *fileName); // Get pointer to extension for a filename string (includes dot: '.png')
const char *GetFileName(const char *filePath); // Get pointer to filename for a path string
const char *GetFileNameWithoutExt(const char *filePath); // Get filename string without extension (uses static string)
const char *GetDirectoryPath(const char *filePath); // Get full path for a given fileName with path (uses static string)
const char *GetPrevDirectoryPath(const char *dirPath); // Get previous directory path for a given path (uses static string)
const char *GetWorkingDirectory(void); // Get current working directory (uses static string)
const char *GetApplicationDirectory(void); // Get the directory of the running application (uses static string)
int MakeDirectory(const char *dirPath); // Create directories (including full path requested), returns 0 on success
bool ChangeDirectory(const char *dirPath); // Change working directory, return true on success
bool IsPathFile(const char *path); // Check if a given path is a file or a directory
bool IsFileNameValid(const char *fileName); // Check if fileName is valid for the platform/OS
FilePathList LoadDirectoryFiles(const char *dirPath); // Load directory filepaths, files and directories, no subdirs scan
FilePathList LoadDirectoryFilesEx(const char *basePath, const char *filter, bool scanSubdirs); // Load directory filepaths with extension filtering and subdir scan; some filters available: " *.*", " "
void UnloadDirectoryFiles(FilePathList files); // Unload filepaths
bool IsFileDropped(void); // Check if a file has been dropped into window
FilePathList LoadDroppedFiles(void); // Load dropped filepaths
void UnloadDroppedFiles(FilePathList files); // Unload dropped filepaths
unsigned int GetDirectoryFileCount(const char *dirPath); // Get the file count in a directory

```

```

unsigned int GetDirectoryFileCountEx(const char *basePath, const char *filter, bool scanSubdirs); // Get the file count in a directory with extension filtering and
// recursive directory scan. Use 'DI'

// Compression/Encoding functionality
unsigned char *CompressData(const unsigned char *data, int dataSize, int *compDataSize); // Compress data (DEFLATE algorithm), memory must be MemFree()
unsigned char *DecompressData(const unsigned char *compData, int compDataSize, int *dataSize); // Decompress data (DEFLATE algorithm), memory must be MemFree()
char *EncodeDataBase64(const unsigned char *data, int dataSize, int *outputSize); // Encode data to Base64 string (includes NULL terminator), memory must be
// MemFree()
unsigned char *DecodeDataBase64(const char *text, int *outputSize); // Decode Base64 string (expected NULL terminated), memory must be
// MemFree()

unsigned int ComputeCRC32(unsigned char *data, int dataSize); // Compute CRC32 hash code
unsigned int *ComputeMD5(unsigned char *data, int dataSize); // Compute MD5 hash code, returns static int[4] (16 bytes)
unsigned int *ComputeSHA1(unsigned char *data, int dataSize); // Compute SHA1 hash code, returns static int[5] (20 bytes)
unsigned int *ComputeSHA256(unsigned char *data, int dataSize); // Compute SHA256 hash code, returns static int[8] (32 bytes)

// Automation events functionality
AutomationEventList LoadAutomationEventList(const char *fileName); // Load automation events list from file, NULL for empty list,
// capacity = MAX_AUTOMATION_EVENTS
void UnloadAutomationEventList(AutomationEventList list); // Unload automation events list from file
bool ExportAutomationEventList(AutomationEventList list, const char *fileName); // Export automation events list as text file
void SetAutomationEventList(AutomationEventList *list); // Set automation event list to record to
void SetAutomationEventBaseFrame(int frame); // Set automation event internal base frame to start recording
void StartAutomationEventRecording(void); // Start recording automation events (AutomationEventList must be set)
void StopAutomationEventRecording(void); // Stop recording automation events
void PlayAutomationEvent(AutomationEvent event); // Play a recorded automation event

//-----
// Input Handling Functions (Module: core)
//-----
// Input-related functions: keyboard
bool IsKeyPressed(int key); // Check if a key has been pressed once
bool IsKeyPressedRepeat(int key); // Check if a key has been pressed again
bool IsKeyDown(int key); // Check if a key is being pressed
bool IsKeyReleased(int key); // Check if a key has been released once
bool IsKeyUp(int key); // Check if a key is NOT being pressed
int GetKeyPressed(void); // Get key pressed (keycode), call it multiple times for keys queued, returns 0 when the queue is empty
int GetCharPressed(void); // Get char pressed (unicode), call it multiple times for chars queued, returns 0 when the queue is empty
const char *GetKeyName(int key); // Get name of a QWERTY key on the current keyboard layout (eg returns string 'q' for KEY_A on an AZERTY keyboard)
void SetExitKey(int key); // Set a custom key to exit program (default is ESC)

// Input-related functions: gamepads
bool IsGamepadAvailable(int gamepad); // Check if a gamepad is available
const char *GetGamepadName(int gamepad); // Get gamepad internal name id
bool IsGamepadButtonPressed(int gamepad, int button); // Check if a gamepad button has been pressed once
bool IsGamepadButtonDown(int gamepad, int button); // Check if a gamepad button is being pressed
bool IsGamepadButtonReleased(int gamepad, int button); // Check if a gamepad button has been released once
bool IsGamepadButtonUp(int gamepad, int button); // Check if a gamepad button is NOT being pressed
int GetGamepadButtonPressed(void); // Get the last gamepad button pressed
int GetGamepadAxisCount(int gamepad); // Get axis count for a gamepad
float GetGamepadAxisMovement(int gamepad, int axis); // Get movement value for a gamepad axis
int SetGamepadMappings(const char *mappings); // Set internal gamepad mappings (SDL_GameControllerDB)
void SetGamepadVibration(int gamepad, float leftMotor, float rightMotor, float duration); // Set gamepad vibration for both motors (duration in seconds)

// Input-related functions: mouse
bool IsMouseButtonPressed(int button); // Check if a mouse button has been pressed once
bool IsMouseButtonDown(int button); // Check if a mouse button is being pressed
bool IsMouseButtonReleased(int button); // Check if a mouse button has been released once
bool IsMouseButtonUp(int button); // Check if a mouse button is NOT being pressed
int GetMouseX(void); // Get mouse position X
int GetMouseY(void); // Get mouse position Y
Vector2 GetMousePosition(void); // Get mouse position XY
Vector2 GetMouseDelta(void); // Get mouse delta between frames
void SetMousePosition(int x, int y); // Set mouse position XY

```

```

void SetMouseOffset(int offsetX, int offsetY); // Set mouse offset
void SetMouseScale(float scaleX, float scaleY); // Set mouse scaling
float GetMouseWheelMove(void); // Get mouse wheel movement for X or Y, whichever is larger
Vector2 GetMouseWheelMoveV(void); // Get mouse wheel movement for both X and Y
void SetMouseCursor(int cursor); // Set mouse cursor

// Input-related functions: touch
int GetTouchX(void); // Get touch position X for touch point 0 (relative to screen size)
int GetTouchY(void); // Get touch position Y for touch point 0 (relative to screen size)
Vector2 GetTouchPosition(int index); // Get touch position XY for a touch point index (relative to screen size)
int GetTouchPointId(int index); // Get touch point identifier for given index
int GetTouchPointCount(void); // Get number of touch points

//-----
// Gestures and Touch Handling Functions (Module: rgestures)
//-----
void SetGesturesEnabled(unsigned int flags); // Enable a set of gestures using flags
bool IsGestureDetected(unsigned int gesture); // Check if a gesture have been detected
int GetGestureDetected(void); // Get latest detected gesture
float GetGestureHoldDuration(void); // Get gesture hold time in seconds
Vector2 GetGestureDragVector(void); // Get gesture drag vector
float GetGestureDragAngle(void); // Get gesture drag angle
Vector2 GetGesturePinchVector(void); // Get gesture pinch delta
float GetGesturePinchAngle(void); // Get gesture pinch angle

//-----
// Camera System Functions (Module: rcamera)
//-----
void UpdateCamera(Camera *camera, int mode); // Update camera position for selected mode
void UpdateCameraPro(Camera *camera, Vector3 movement, Vector3 rotation, float zoom); // Update camera movement/rotation

```

rttextures

// Image loading functions

// NOTE: These functions do not require GPU access

```
Image LoadImage(const char *fileName);
Image LoadImageRaw(const char *fileName, int width, int height, int format, int headerSize);
Image LoadImageAnim(const char *fileName, int *frames);

Image LoadImageAnimFromMemory(const char *fileType, const unsigned char *fileData, int dataSize, int *frames);
Image LoadImageFromMemory(const char *fileType, const unsigned char *fileData, int dataSize);

Image LoadImageFromTexture(Texture2D texture);
Image LoadImageFromScreen(void);
bool IsImageValid(Image image);
void UnloadImage(Image image);
bool ExportImage(Image image, const char *fileName);
unsigned char *ExportImageToMemory(Image image, const char *fileType, int *fileSize);
bool ExportImageAsCode(Image image, const char *fileName);

// Load image from file into CPU memory (RAM)
// Load image from RAW file data
// Load image sequence from file (frames appended to
// image.data)
// Load image sequence from memory buffer
// Load image from memory buffer, fileType refers to
// extension: i.e. '.png'
// Load image from GPU texture data
// Load image from screen buffer and (screenshot)
// Check if an image is valid (data and parameters)
// Unload image from CPU memory (RAM)
// Export image data to file, returns true on success
// Export image to memory buffer, memory must be MemFree()
// Export image as code file defining an array of bytes,
// returns true on success
```

// Image generation functions

```
Image GenImageColor(int width, int height, Color color);
Image GenImageGradientLinear(int width, int height, int direction, Color start, Color end);

Image GenImageGradientRadial(int width, int height, float density, Color inner, Color outer);
Image GenImageGradientSquare(int width, int height, float density, Color inner, Color outer);
Image GenImageChecked(int width, int height, int checksX, int checksY, Color col1, Color col2);
Image GenImageWhiteNoise(int width, int height, float factor);
Image GenImagePerlinNoise(int width, int height, int offsetX, int offsetY, float scale);
Image GenImageCellular(int width, int height, int tileSize);
Image GenImageText(int width, int height, const char *text);

// Generate image: plain color
// Generate image: linear gradient, direction in degrees [0..360],
// 0=Vertical gradient
// Generate image: radial gradient
// Generate image: square gradient
// Generate image: checked
// Generate image: white noise
// Generate image: perlin noise
// Generate image: cellular algorithm, bigger tileSize means bigger cells
// Generate image: grayscale image from text data
```

// Image manipulation functions

```
Image ImageCopy(Image image);
Image ImageFromImage(Image image, Rectangle rec);
Image ImageFromChannel(Image image, int selectedChannel);

Image ImageText(const char *text, int fontSize, Color color);
Image ImageTextEx(Font font, const char *text, float fontSize, float spacing, Color tint);
void ImageFormat(Image *image, int newFormat);
void ImageToPOT(Image *image, Color fill);
void ImageCrop(Image *image, Rectangle crop);
void ImageAlphaCrop(Image *image, float threshold);
void ImageAlphaClear(Image *image, Color color, float threshold);
void ImageAlphaMask(Image *image, Image alphaMask);
void ImageAlphaPremultiply(Image *image);
void ImageBlurGaussian(Image *image, int blurSize);
void ImageKernelConvolution(Image *image, const float *kernel, int kernelSize);
void ImageResize(Image *image, int newWidth, int newHeight);
void ImageResizeNN(Image *image, int newWidth, int newHeight);
void ImageResizeCanvas(Image *image, int newWidth, int newHeight, int offsetX, int offsetY, Color fill);
void ImageMipmaps(Image *image);
void ImageDither(Image *image, int rBpp, int gBpp, int bBpp, int aBpp);

void ImageFlipVertical(Image *image);
void ImageFlipHorizontal(Image *image);
void ImageRotate(Image *image, int degrees);
void ImageRotateCW(Image *image);
void ImageRotateCCW(Image *image);
void ImageColorTint(Image *image, Color color);
void ImageColorInvert(Image *image);
void ImageColorGrayscale(Image *image);
void ImageColorContrast(Image *image, float contrast);
void ImageColorBrightness(Image *image, int brightness);
void ImageColorReplace(Image *image, Color color, Color replace);

// Create an image duplicate (useful for transformations)
// Create an image from another image piece
// Create an image from a selected channel of another image
// (GRAYSCALE)
// Create an image from text (default font)
// Create an image from text (custom sprite font)
// Convert image data to desired format
// Convert image to POT (power-of-two)
// Crop an image to a defined rectangle
// Crop image depending on alpha value
// Clear alpha channel to desired color
// Apply alpha mask to image
// Premultiply alpha channel
// Apply Gaussian blur using a box blur approximation
// Apply custom square convolution kernel to image
// Resize image (Bicubic scaling algorithm)
// Resize image (Nearest-Neighbor scaling algorithm)
// Resize canvas and fill with color
// Compute all mipmap levels for a provided image
// Dither image data to 16bpp or lower (Floyd-Steinberg
// dithering)
// Flip image vertically
// Flip image horizontally
// Rotate image by input angle in degrees (-359 to 359)
// Rotate image clockwise 90deg
// Rotate image counter-clockwise 90deg
// Modify image color: tint
// Modify image color: invert
// Modify image color: grayscale
// Modify image color: contrast (-100 to 100)
// Modify image color: brightness (-255 to 255)
// Modify image color: replace color
```

```
Color *LoadImageColors(Image image);
Color *LoadImagePalette(Image image, int maxPaletteSize, int *colorCount);
```

```
void UnloadImageColors(Color *colors);
void UnloadImagePalette(Color *colors);
Rectangle GetImageAlphaBorder(Image image, float threshold);
Color GetImageColor(Image image, int x, int y);
```

// Image drawing functions

// NOTE: Image software-rendering functions (CPU)

```
void ImageClearBackground(Image *dst, Color color);
void ImageDrawPixel(Image *dst, int posX, int posY, Color color);
void ImageDrawPixelV(Image *dst, Vector2 position, Color color);
void ImageDrawLine(Image *dst, int startPosX, int startPosY, int endPosX, int endPosY, Color color);
void ImageDrawLineV(Image *dst, Vector2 start, Vector2 end, Color color);
void ImageDrawLineEx(Image *dst, Vector2 start, Vector2 end, int thick, Color color);

void ImageDrawCircle(Image *dst, int centerX, int centerY, int radius, Color color);
void ImageDrawCircleV(Image *dst, Vector2 center, int radius, Color color);

void ImageDrawCircleLines(Image *dst, int centerX, int centerY, int radius, Color color);
void ImageDrawCircleLinesV(Image *dst, Vector2 center, int radius, Color color);

void ImageDrawRectangle(Image *dst, int posX, int posY, int width, int height, Color color);
void ImageDrawRectangleV(Image *dst, Vector2 position, Vector2 size, Color color);

void ImageDrawRectangleRec(Image *dst, Rectangle rec, Color color);
void ImageDrawRectangleLines(Image *dst, Rectangle rec, int thick, Color color);
void ImageDrawTriangle(Image *dst, Vector2 v1, Vector2 v2, Vector2 v3, Color color);
void ImageDrawTriangleEx(Image *dst, Vector2 v1, Vector2 v2, Vector2 v3, Color c1, Color c2, Color c3);

void ImageDrawTriangleLines(Image *dst, Vector2 v1, Vector2 v2, Vector2 v3, Color color);
void ImageDrawTriangleFan(Image *dst, const Vector2 *points, int pointCount, Color color);

void ImageDrawTriangleStrip(Image *dst, const Vector2 *points, int pointCount, Color color);

void ImageDraw(Image *dst, Image src, Rectangle srcRec, Rectangle dstRec, Color tint);

void ImageDrawText(Image *dst, const char *text, int posX, int posY, int fontSize, Color color);

void ImageDrawTextEx(Image *dst, Font font, const char *text, Vector2 position, float fontSize, float spacing, Color tint);
```

```
// Load color data from image as a Color array (RGBA - 32bit)
// Load colors palette from image as a Color array (RGBA -
// 32bit)
// Unload color data loaded with LoadImageColors()
// Unload colors palette loaded with LoadImagePalette()
// Get image alpha border rectangle
// Get image pixel color at (x, y) position
```

```
// Clear image background with given color
// Draw pixel within an image
// Draw pixel within an image (Vector version)
// Draw line within an image
// Draw line within an image (Vector version)
// Draw a line defining thickness within an
// image
// Draw a filled circle within an image
// Draw a filled circle within an image (Vector
// version)
// Draw circle outline within an image
// Draw circle outline within an image (Vector
// version)
// Draw rectangle within an image
// Draw rectangle within an image (Vector
// version)
// Draw rectangle within an image
// Draw rectangle lines within an image
// Draw triangle within an image
// Draw triangle with interpolated colors
// within an image
// Draw triangle outline within an image
// Draw a triangle fan defined by points within
// an image (first vertex is the center)
// Draw a triangle strip defined by points
// within an image
// Draw a source image within a destination
// image (tint applied to source)
// Draw text (using default font) within an
// image (destination)
// Draw text (custom sprite font) within an
// image (destination)
```

// Texture loading functions

// NOTE: These functions require GPU access

```
Texture2D LoadTexture(const char *fileName); // Load texture from file into GPU memory (VRAM)
Texture2D LoadTextureFromImage(Image image); // Load texture from image data
TextureCubemap LoadTextureCubemap(Image image, int layout); // Load cubemap from image, multiple image cubemap layouts supported
RenderTexture2D LoadRenderTexture(int width, int height); // Load texture for rendering (framebuffer)
bool IsTextureValid(Texture2D texture); // Check if a texture is valid (loaded in GPU)
void UnloadTexture(Texture2D texture); // Unload texture from GPU memory (VRAM)
bool IsRenderTextureValid(RenderTexture2D target); // Check if a render texture is valid (loaded in GPU)
void UnloadRenderTexture(RenderTexture2D target); // Unload render texture from GPU memory (VRAM)
void UpdateTexture(Texture2D texture, const void *pixels); // Update GPU texture with new data (pixels should be able to fill texture)
void UpdateTextureRec(Texture2D texture, Rectangle rec, const void *pixels); // Update GPU texture rectangle with new data (pixels and rec should fit in texture)
```

// Texture configuration functions

```
void GenTextureMipmaps(Texture2D *texture); // Set texture scaling filter mode
void SetTextureFilter(Texture2D texture, int filter); // Set texture wrapping mode
void SetTextureWrap(Texture2D texture, int wrap); // Generate GPU mipmaps for a texture
```

```

// Texture drawing functions
void DrawTexture(Texture2D texture, int posX, int posY, Color tint);
void DrawTextureV(Texture2D texture, Vector2 position, Color tint);

void DrawTextureEx(Texture2D texture, Vector2 position, float rotation, float scale, Color tint);

void DrawTextureRec(Texture2D texture, Rectangle source, Vector2 position, Color tint);

void DrawTexturePro(Texture2D texture, Rectangle source, Rectangle dest, Vector2 origin, float rotation, Color tint);

void DrawTextureNPatch(Texture2D texture, NPatchInfo nPatchInfo, Rectangle dest, Vector2 origin, float rotation, Color tint);

// Color/pixel related functions
bool ColorIsEqual(Color col1, Color col2);
Color Fade(Color color, float alpha);
int ColorToInt(Color color);
Vector4 ColorNormalize(Color color);
Color ColorFromNormalized(Vector4 normalized);
Vector3 ColorToHSV(Color color);
Color ColorFromHSV(float hue, float saturation, float value);
Color ColorTint(Color color, Color tint);
Color ColorBrightness(Color color, float factor);
Color ColorContrast(Color color, float contrast);
Color ColorAlpha(Color color, float alpha);

// Check if two colors are equal
// Get color with alpha applied, alpha goes from 0.0f to 1.0f
// Get hexadecimal value for a Color (0xRRGGBBAA)
// Get Color normalized as float [0..1]
// Get Color from normalized values [0..1]
// Get HSV values for a Color, hue [0..360], saturation/value [0..1]
// Get a Color from HSV values, hue [0..360], saturation/value [0..1]
// Get color multiplied with another color
// Get color with brightness correction, brightness factor goes from -1.0f to 1.0f
// Get color with contrast correction, contrast values between -1.0f and 1.0f
// Get color with alpha applied, alpha goes from 0.0f to 1.0f

// Draw a Texture2D
// Draw a Texture2D with position defined
// as Vector2
// Draw a Texture2D with extended
// parameters
// Draw a part of a texture defined by a
// rectangle
// Draw a part of a texture defined by a
// rectangle with 'pro' parameters
// Draws a texture (or part of it) that
// stretches or shrinks nicely

```